Foundations of Long short-term memory

[Long short-term memory]

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$h_t = o_t \odot \tanh(c_t)$$

1.0 Content Block

Each of the gates can be thought as a "standard" neuron in a feed-forward (or multi-layer) neural network: that is, they compute an activation (using an activation function) of a weighted sum. i t , o t {\displaystyle i_{t},o_{t}} and f t {\displaystyle f_{t}} represent the activations of respectively the input, output and forget gates, at time step t {\displaystyle t} . The 3 exit arrows from the memory cell c {\displaystyle c} to the 3 gates i , o {\displaystyle i,o} and f {\displaystyle f} represent the peephole connections. These peephole connections actually denote the contributions of the activation of the memory cell c {\displaystyle c} at time step t – 1 {\displaystyle t-1} , i.e. the contribution of c t – 1 {\displaystyle c_{t-1}} (and not c t {\displaystyle c_{t}} , as the picture may suggest). In other words, the gates i , o {\displaystyle i,o} and f {\displaystyle f} calculate their activations at time step t {\displaystyle t} (i.e., respectively, i t , o t {\displaystyle i_{t},o_{t}} and f t {\displaystyle f_{t}}) also considering the activation of the memory cell c {\displaystyle c} at time step t – 1 {\displaystyle t-1} , i.e. c t – 1 {\displaystyle c_{t-1}} . The single left-to-right arrow exiting the memory cell is not a peephole connection and denotes c t {\displaystyle c_{t}} . The little circles containing a × {\displaystyle \times } symbol represent an element-wise multiplication between its inputs. The big circles containing an S-like curve represent the application of a differentiable function (like the sigmoid function) to a weighted sum.

2.0 Content Block

Motivation In theory, classic RNNs can keep track of arbitrary long-term dependencies in the input sequences. The problem with classic RNNs is computational (or practical) in nature: when training a classic RNN using back-propagation, the long-term gradients which are back-propagated can "vanish", meaning they can tend to zero due to very small numbers creeping into the computations, causing the model to effectively stop learning. RNNs using LSTM units partially solve the vanishing gradient problem, because LSTM units allow gradients to also flow with little to no attenuation. However, LSTM networks can still suffer from the exploding gradient problem. The intuition behind the LSTM architecture is to create an additional module in a neural network that learns when to remember and when to forget pertinent information. In other words, the network effectively learns which information might be needed later on in a sequence and when that information is no longer needed. For instance, in the context of natural language processing, the network can learn grammatical dependencies. An LSTM might process the sentence "Dave, as a result of his controversial claims, is now a pariah" by remembering the (statistically likely) grammatical gender and number of the subject Dave. Note that this information is pertinent for the pronoun his and note that this information is no longer important after the verb is.

3.0 Content Block

f t = σ g (W f x t + U f c t – 1 + b f) i t = σ g (W i x t + U i c t – 1 + b i) o t = σ g (W o x t + U o c t – 1 + b o) c t = f t ■ c t – 1 + i t ■ σ c (W c x t + b c) h t = o t ■ σ h (c t) {\displaystyle {\begin{aligned}f_{t}&=\sigma

_{g}(W_{f}x_{t}+U_{f}c_{t-1}+b_{f})\\i_{t}&=\sigma _{g}(W_{i}x_{t}+U_{i}c_{t-1}+b_{i})\\o_{t}&=\sigma _{g}(W_{o}x_{t}+U_{o}c_{t-1}+b_{o})\\c_{t}&=f_{t}\odot c_{t-1}+i_{t}\odot \sigma _{c}(W_{c}x_{t}+b_{c})\\h_{t}&=o_{t}\odot \sigma _{h}(c_{t})\end{aligned}}}

4.0 Content Block

Further reading Monner, Derek D.; Reggia, James A. (2010). "A generalized LSTM-like training algorithm for second-order recurrent neural networks" (PDF). Neural Networks. 25 (1): 70–83. doi:10.1016/j.neunet.2011.07.003. PMC 3217173. PMID 21803542. High-performing extension of LSTM that has been simplified to a single node type and can train arbitrary architectures Gers, Felix A.; Schraudolph, Nicol N.; Schmidhuber, Jürgen (Aug 2002). "Learning precise timing with LSTM recurrent networks" (PDF). Journal of Machine Learning Research. 3: 115–143. Gers, Felix (2001). "Long Short-Term Memory in Recurrent Neural Networks" (PDF). PhD thesis. Abidogun, Olusola Adeniyi (2005). Data Mining, Fraud Detection and Mobile Telecommunications: Call Pattern Analysis with Unsupervised Neural Networks. Master's Thesis (Thesis). University of the Western Cape. hdl:11394/249. Archived (PDF) from the original on May 22, 2012. original with two chapters devoted to explaining recurrent neural networks, especially LSTM.

5.0 Content Block

The figure on the right is a graphical representation of an LSTM unit with peephole connections (i.e. a peephole LSTM). Peephole connections allow the gates to access the constant error carousel (CEC), whose activation is the cell state. h t – 1 {\displaystyle h_{t-1}} is not used, c t – 1 {\displaystyle c_{t-1}} is used instead in most places.

6.0 Content Block

Development of variants In 2005 Graves and Schmidhuber published LSTM with full backpropagation through time and bidirectional LSTM. In 2006 Graves, Fernandez, Gomez, and Schmidhuber introduced a new error function for LSTM: Connectionist Temporal Classification (CTC) for simultaneous alignment and recognition of sequences. In 2014 Kyunghyun Cho an others published a simplified variant of the forget gate LSTM called Gated recurrent unit (GRU). In 2015 Srivastava, Greff, and Schmidhuber used LSTM principles to create the Highway network, a feedforward neural network with hundreds of layers, much deeper than previous networks. Concurrently, the ResNet architecture was developed. It is equivalent to an open-gated or gateless highway network. A modern upgrade of LSTM called xLSTM is published by a team led by Sepp Hochreiter. One of the 2 blocks (mLSTM) of the architecture are parallelizable like the Transformer architecture, the other ones (sLSTM) allow state tracking.

7.0 Content Block

Training An RNN using LSTM units can be trained in a supervised fashion on a set of training sequences, using an optimization algorithm like gradient descent combined with backpropagation through time to compute the gradients needed during the optimization process, in order to change each weight of the LSTM network in proportion to the derivative of the error (at the output layer of the LSTM network) with respect to corresponding weight. A problem with using gradient descent for standard RNNs is that error gradients vanish exponentially quickly with the size of the time lag between important events. This is due to lim n → ∞ W n = 0 {\displaystyle \lim _{n\to \infty }W^{n}=0} if the spectral radius of W {\displaystyle W} is smaller than 1. However, with LSTM units, when error values are back-propagated from the output layer, the error remains in the LSTM unit's cell. This "error carousel" continuously feeds error back to each of the LSTM unit's gates, until they learn to cut off the value.